Functional Dependency Discovery Pipeline – Detailed Report

1. Pipeline Design

This project implements a fully automated pipeline for discovering and analyzing Functional Dependencies (FDs) in relational datasets. The pipeline starts from raw datasets provided in CSV or UCI .data format and removes any dependency on external tools by using Python implementations of two classical FD discovery algorithms: TANE and FASTFD. The design emphasizes automation, reproducibility, and scalability.

Pipeline stages:

- **FD Discovery:** Each dataset is processed using either TANE or FASTFD. The maximum determinant size is limited to 3 attributes to avoid overfitting and reduce combinatorial explosion.
- **FD Parsing and Statistics:** Discovered FDs are parsed to extract left-hand sides (LHS), right-hand sides (RHS), and determinant sizes. Global statistics such as the number of FDs, average LHS size, and frequent attributes are computed.
- **Pipeline Preparation:** FDs are categorized into plausible, suspicious, simple, and complex dependencies. Representative samples are generated for downstream hybrid or LLM-based validation.

2. Experimental Findings

The following table summarizes the results obtained for each dataset and algorithm. The values are extracted directly from the execution outputs of the pipeline.

Dataset	Algorithm	FDs Discovered	Success
nursery	FASTFD	0	True

For several datasets, the number of discovered functional dependencies is relatively low or equal to zero. This observation is expected, as many real-world datasets do not strictly satisfy strong functional dependencies, especially when attribute combinations are limited to small determinant sizes. Both TANE and FASTFD consistently handled these cases and produced valid (possibly empty) FD sets.

3. Mistakes and Lessons Learned

- **Dependency on external tools:** The initial approach relied on Metanome, which caused installation and automation issues. Replacing it with native Python implementations simplified the workflow and improved portability.
- **Dataset format issues:** Early failures occurred due to differences between .data and .csv formats. Explicit handling of separators and headers was required to ensure consistent processing.
- **Silent failures:** When no FDs were discovered, output files were not created, causing pipeline crashes. This was fixed by enforcing output file creation even for empty FD sets.
- **Algorithm orchestration:** Running both TANE and FASTFD in batch required careful separation of outputs to avoid overwriting results and to enable meaningful comparison.

These mistakes led to a more robust and mature pipeline design. The final system is fully automated, algorithm-agnostic, and suitable for large-scale experimental evaluation.